SymCheck

Anonymous Author(s)

Abstract

ACM Reference Format:
Anonymous Author(s). 2026. SymCheck. In . ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

2 Motivating Example

3 Background

4 Design

4.1 The Surface Language

(1) Π is our path, or "context." It keeps track of which labels are taking on which values currently.
(2) Γ is our environment. It functions as a dictionary that maps labels to values. Since a single variable may take on multiple values in different contexts, the environment really maps labels to dictionaries which map contexts to values.
(3) Δ is our dependency map. Since certain variable are only defined in the context of other variables, we need a map to tell us which other variable must be defined to bring another variable into scope. So the dependency maps labels to lists of labels for this purpose. For instance, consider the symbolic expression $a \rightarrow (b \rightarrow b)*_a$. Since b is defined in the star expression labelled "a" then b is only defined if a is defined in the current context of an expression. Thus, $\Delta(b) = [a]$. When using the environment, we assume the dependency map is precomputed. We also assume the dependency map is consulted before looking up a variables specific value in the environment.
(4) $E[\ell \rightarrow x]$ means "Insert the key-value pair (ℓ, x) into E." if E is Γ then we assume that this operation takes the current context and dependency map into account
(5) $E(\ell)$ means "get the value for key x in E." If E is Γ then we assume that this operation takes the current context and dependency map into account.
(6) $\ell \in E$ means "check if ℓ is in the key set of E"
(7) $x : T$ means "x is of type T" where T is either a base type β, an arbitrary type τ, or. A base type is a type that is not a list, an arbitrary type either a base type or a list of some arbitrary type. TODO, this isn't quite true I don't think... it really means "surface expression x will generate values of type T." maybe need new symbol for this?

$$
\begin{aligned}
surface \rightarrow\ & [surface] \\
|\ & surface\ '|'\ surface \\
|\ & surface\ '|\{\ell\}'\ surface \\
|\ & surface* \\
|\ & surface * \{\ell\} \\
|\ & surface\ surface \\
|\ & \ell \mapsto surface \\
|\ & (\ell \Longleftarrow b) \mapsto surface \\
|\ & \ell \\
|\ & . \\
|\ & (. \Longleftarrow b) \\
|\ & < x > \\
|\ & [\ ]
\end{aligned}
$$

Figure 1: The Surface Language as a Context Free Grammar

(8) $e \Downarrow x$ means "symbolic expression e generates value x"
(9) $x \longrightarrow y$ means "surface expression x translates to core expression y"
(10) $xs + +ys$ means "append the list ys to the list xs"
(11) $x :: xs$ means "prepend the element x to the list xs"
(12) $x\ fresh$ means that value x is a random value that will never be used again in any other expression
(13) $x\ arbitrary$ means that value x is a random value that may be used again in subsequent expressions.
(14) $rand()$ is a function that generates and arbitrary value of the correct type (context dependent) and returns it.

The surface language is the language that the user interacts with. Their expressions are defined in terms of the surface language so they can be translated to the core language. In this specification, ℓ is a label (i.e a string), x is any arbitrary expression, and b is some expression that evaluates to a boolean. The b expression exists so that the user can constrain the domains of variables so that the generation process isn't completely random. If b is just "$True$" then SymCheck will use the first value it generates for the label. It is possible, however, to include a predicate here which can include surface expressions as arguments so that labels can be constrained in terms of other labels. If this is done, it is assumed that the arguments are translated to the core language (see section 4.3) and evaluated (see section 4.7) before being passed to the predicate.

4.2 The Core Language

After translation all expressions are in the core language. The core language provides better typechecking for the compiler.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

$$
\begin{aligned}
core \rightarrow\ & core' |'_{\ell} core \\
& \mid core *_{\ell} \\
& \mid core :> core \\
& \mid core :++ core \\
& \mid (\ell \Longleftarrow b) \mapsto core \\
& \mid \ell \\
& \mid\ < x > \\
& \mid [\ ]
\end{aligned}
$$

Figure 2: The Core Language as a Context Free Grammar

4.3 Translation Rules

4.4 Dependency Map Computation

the dependency map function is a bit complex and I'm not sure if we need rules for it anyways or if its sufficient to just describe the algorithm. Since we build the dependency map from a whole series of expressions representing a function I designed it so that it could be a fold function (i.e its type is dmap -> ast n a -> dmap)

(1) TODO: rules go here

4.5 Scope Checking Rules

The scope checking rules aim to check if every label used in a tree is used in an appropriate place which really boils down to two rules

(1) No label is used without first being declared
(2) No label is used unless the dependency map allows it

(1) TODO: add rules

4.6 Well Formedness Rules

The well formedness rules aim to make sure that the depths of each expression and subexpression match the rules of our Ast n a GADT (this may already be done in the translation rules?)

(1) TODO: add rules

4.7 Generation Rules

The generation rules demonstrate how a nested list is generated from a symbolic expression

(1) $\text{Variable}\ \dfrac{\Gamma(\ell) = x}{(\Gamma, \Pi)\ \ell \Downarrow x,\ \Gamma}$

(2) $\text{Empty}\ \dfrac{}{(\Gamma, \Pi)\ [\ ] \Downarrow [\ ],\ \Gamma}$

(3) $\text{UnionLeft}\ \dfrac{\Gamma(\ell)\ \ \text{mod}\ 2 = 0 \qquad (\Gamma, \Pi[\ell \rightarrow 0])\ r_1 \Downarrow x,\ \Gamma'}{(\Gamma, \Pi)\ r_1 |_{\ell} r_2 \Downarrow x,\ \Gamma'}$

(4) $\text{UnionRight}\ \dfrac{\Gamma(\ell)\ \ \text{mod}\ 2 = 1 \qquad (\Gamma, \Pi[\ell \rightarrow 0])\ r_2 \Downarrow x,\ \Gamma'}{(\Gamma, \Pi)\ r_1 |_{\ell} r_2 \Downarrow x,\ \Gamma'}$

(5) $\text{StarInit}\ \dfrac{\ell \notin \Pi \qquad \Gamma(\ell) = n \qquad (\Gamma, \Pi[\ell \rightarrow 0])\ r *_{\ell} \Downarrow xs,\ \Gamma'}{(\Gamma, \Pi)\ r *_{\ell} \Downarrow xs,\ \Gamma'}$

(6) $\text{Star}\ \dfrac{\Pi(\ell) < n \qquad \Gamma(\ell) = n \qquad (\Gamma, \Pi)\ r \Downarrow xs,\ \Gamma' \qquad (\Gamma, \Pi[\ell \rightarrow \Pi(\ell) + 1])\ r *_{\ell} \Downarrow ys}{(\Gamma, \Pi)\ r *_{\ell} \Downarrow xs + +ys}$

(7) $\text{StarEnd}\ \dfrac{\Gamma(\ell) \leq 0 \vee \Pi(\ell) \geq \Gamma(\ell)}{(\Gamma, \Pi)\ r *_{\ell} \Downarrow [\ ],\ \Gamma'}$

(8) $\text{Concat}\ \dfrac{(\Gamma, \Pi)\ r_1 \Downarrow xs,\ \Gamma' \qquad r_2 \Downarrow ys,\ \Gamma''}{(\Gamma, \Pi)\ r_1 :++ r_2 \Downarrow xs + +ys,\ \Gamma''}$

(9) $\text{Cons}\ \dfrac{(\Gamma, \Pi)\ r_1 \Downarrow x,\ \Gamma' \qquad \Gamma' r_1 \Downarrow xs,\ \Gamma''}{(\Gamma, \Pi)\ r_1 :> r_2 \Downarrow x : xs,\ \Gamma''}$

(10) $\text{Literals}\ \dfrac{}{(\Gamma, \Pi)\ \ < c > \Downarrow c,\ \Gamma'}$

(11) $\text{Apply}\ \dfrac{(\Gamma, \Pi)\ \ args \Downarrow args' \Gamma}{(\Gamma, \Pi)\ \ < f\ \ args > \Downarrow f(args'), \Gamma}$

(12) $\text{Assign}\ \dfrac{x\ \ arbitrary \qquad p(args, x)}{(\Gamma, \Pi)\ (\ell \Longleftarrow p(args, \ell)) \mapsto e,\ \Gamma[\ell \rightarrow x]}$

4.8 Check Rules

The check rules aim to determine if it is possible that a given nested list could be generated by a given ast. I'm gonna need some special syntax here that means "there exists a prefix of this list such that..." and "and the rest of the list has some other property" for concat, star, and maybe wrap/cons

(1) TODO: add rules

5 Implementation

6 Case Studies

7 Conclusion

$$\text{AppTrans}\ \frac{\forall arg_i \in args,}{< f\ args > \longrightarrow < f\ args' >}$$

$$\text{UnionTrans}\ \frac{x : \tau \quad y : \tau \quad x \longrightarrow x' \quad y \longrightarrow y'}{x|\{\ell\}y \longrightarrow x' \mid_{\ell} y'}$$

$$\text{UnlabelledUnionTrans}\ \frac{x : \tau \quad y : \tau \quad x \longrightarrow x' \quad y \longrightarrow y'}{x \mid y \longrightarrow (\ell' \Longleftarrow True) \mapsto x' \mid_{\ell'}}$$

$$\text{StarBaseTrans}\ \frac{x : \beta \quad x \longrightarrow x'}{x * \{\ell\} \longrightarrow [x']*_{\ell}}$$

$$\text{StarTrans}\ \frac{x : [\tau] \quad x \longrightarrow x'}{x * \{\ell\} \longrightarrow x'*_{\ell}}$$

$$\text{UnlabelledStarBaseTrans}\ \frac{x : \beta \quad x \longrightarrow x' \quad \ell'\ fresh}{x* \longrightarrow (\ell' \Longleftarrow True) \mapsto [x']*_{\ell'}}$$

$$\text{UnlabelledStarTrans}\ \frac{x : [\tau] \quad x \longrightarrow x' \quad \ell'\ fresh}{x* \longrightarrow (\ell' \Longleftarrow True) \mapsto x'*_{\ell'}}$$

$$\text{ConcatBaseTrans}\ \frac{x : \beta \quad y : \beta \quad x \longrightarrow x' \quad y \longrightarrow y'}{x\ y \longrightarrow x' :> [y']}$$

$$\text{ConcatConsTrans}\ \frac{x : \tau \quad y : [\tau] \quad x \longrightarrow x' \quad y \longrightarrow y'}{x\ y \longrightarrow x' :> y'}$$

$$\text{ConcatSnocTrans}\ \frac{x : [\tau] \quad y : \tau \quad x \longrightarrow x' \quad y \longrightarrow y'}{x\ y \longrightarrow x' : ++ [y']}$$

$$\text{ConcatTrans}\ \frac{x : [\tau] \quad y : [\tau] \quad x \longrightarrow x' \quad y \longrightarrow y'}{x\ y \longrightarrow x' : ++ y'}$$

$$\text{AssignmentWithPredTrans}\ \frac{\forall arg \in args, arg \longrightarrow arg' \quad x \longrightarrow x'}{(\ell \Longleftarrow p(args, \ell)) \mapsto x \longrightarrow (\ell \Longleftarrow p(args', \ell)) \mapsto x'}$$

$$\text{AssignmentNoPredTrans}\ \frac{x \longrightarrow x'}{\ell \mapsto x \longrightarrow (\ell \Longleftarrow True) \mapsto x'}$$

$$\text{Variable}\ \frac{}{\ell \longrightarrow \ell}$$

$$\text{DotWithPredTrans}\ \frac{args * \longrightarrow args' \quad fresh(\ell')}{(. \Longleftarrow p\ args) \longrightarrow (\ell' \Longleftarrow p(args', \ell)) \mapsto \ell'}$$

$$\text{DotNoPredTrans}\ \frac{(\ fresh\ell')}{. \longrightarrow (\ell' \Longleftarrow True) \mapsto \ell'}$$

$$\text{LitTrans}\ \frac{}{< c > \longrightarrow < c >}$$

$$\text{EmptyTrans}\ \frac{}{[\ ] \longrightarrow [\ ]}$$

Figure 3